

Trekio - Aplicacao de Trilhos e Caminhadas

João Pedro Grave Martins
Tiago de Bragança Praia
Afonso Duarte de Freitas e Sá Lopes

Orientadores: Paulo Alexandre Leal Barros Pereira
Pedro Alexandre de Seia e Cunha Ribeiro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Trekio - Aplicação de Trilhos e Caminhadas

50055 João Pedro Grave Martins

51497 Tiago de Bragança Praia

51498 Afonso Duarte de Freitas e Sá Lopes

Orientadores: Paulo Alexandre Leal Barros Pereira

Pedro Alexandre de Seia e Cunha Ribeiro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

Resumo

Os caminhantes enfrentam um problema geral ao planejar as suas atividades: encontrar um ou mais trilhos adequados aos seus interesses. Esta é a principal motivação para o desenvolvimento deste projeto. O objetivo deste projeto é criar uma aplicação, Trekio, focada na acumulação de informação sobre possíveis trilhos, que os utilizadores possam ter interesse em percorrer, ou ganharem o conhecimento da existência deles. Foi então projetado como componentes principais um servidor, onde são guardadas todas as informações sobre os utilizadores, trilhos e caminhadas exercidas pelos utilizadores, e uma aplicação mobile para Android. Também foi projetado a possibilidade de execução em Web, não tendo possibilidade de executar em iOS por falta de dispositivos deste sistema.

A aplicação fornece, em tempo real, várias sugestões de trilhos ao utilizador com base no seu pedido (através de parâmetros definidos). Com base nos trilhos anteriormente realizados pelo utilizador, poderão ser sugeridas novas rotas semelhantes ou rotas já efetuadas, ao mesmo tempo que será disponibilizado um histórico para consulta futura. É possível mudar a linguagem, tema e métrica dentro da aplicação para que o utilizador possa ter uma experiência mais pessoal.

Palavras-Chave: Trilhos, Caminhada, Utilizador, Mapa, Kotlin Multiplatform

Utilizacao de Inteligência Arti cial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa como apoio no desenvolvimento de código e na necessidade de resolução de problemas. Foram utilizadas ferramentas como Claude, Gemini e GitHub Copilot, sempre com o objetivo de otimizar a criação de código e fazer análises de documentação mais complexa sobre algumas bibliotecas utilizadas. Alguns exemplos são:

- Bibliotecas Mapbox Android e Mapbox GL JS, utilizadas para instanciar os mapas em Android e Web.
- Animações de Interface de Utilizador.
- Cálculos matemáticos complexos.
- Testes Unitários.

Alterações feitas apos entrega

Esta versão do relatório inclui as seguintes alterações relativamente à versão final submetida. Alterações gramaticais foram feitas pontualmente, não sendo incluídas nesta lista. Secções irão aparecer em páginas diferentes devido a estas alterações:

1. Alteração do Estado de arte, por ter ficado muito superficial, não indicando soluções da área. (ver secção 1.2)
2. Mínima alteração nos objetivos e metas, indicando logo que não foi possível introduzir a funcionalidade com LLM na aplicação . (ver secção 1.3)
3. Adicionada a secção de Organização de Relatório (ver secção 1.5)
4. Corrigida a descrição de JWT e Bearer na secção de Protocolos: JWT passa a ser identificado como o formato do token de acesso, e Bearer como o esquema de transmissão de ambos os tokens (acesso e atualização), e não apenas do token de atualização. (ver secção 2.2)
5. Alterada a explicação (e localização, agora antes de Backend, na secção Software) da tecnologia Ktor, por estar incompleta: não distinguia a utilização no *frontend* (*Ktor Client*) da utilização no *backend* (*Ktor Server*), nem os diferentes motores escolhidos para cada caso. (ver secção 3.2)
6. Alterada a explicação da tabela OAuthCodes, no Modelo ER, pois foi possível resolver o problema indicado sobre "email" e "username" não serem chaves estrangeiras. (ver secção 4.3.2.4)
7. Adicionada uma explicação sobre a arquitetura do projeto. (ver secção 4.1)
8. Alterada a explicação do Ecra de Caminhada, por ter ficado superficial. (ver secção 4.2.2.7)
9. Foi adicionado uma explicação sobre a utilização do Docker e sobre Automação do Ambiente. (ver secção 4.3.5)
10. Alterada a conclusão, por ter ficado muito superficial, não tendo sido incluído explicação dos desafios, aprendizagens e funcionalidades não implementadas (ver secção 5)

Índice

Lista de Figuras	x
Lista de Acrónimos	xiii
1 Introdução	1
1.1 Enquadramento	1
1.2 Estado da arte	1
1.3 Objetivos e metas	2
1.4 Resumo da solução proposta	2
1.5 Organização do relatório	2
2 Tecnologias e Protocolos	3
2.1 Tecnologias Adotadas	3
2.2 Protocolos	4
3 Software	5
3.1 Frontend	5
3.1.1 Tecnologia para Alvos	5
3.1.2 Biblioteca de Mapa	6
3.2 Ktor	7
3.2.1 Ktor Client	7
3.2.2 Ktor Server	8
3.3 Backend	9
3.3.1 Segurança	9
3.3.2 Tratamento de Erros	10
3.3.3 Acesso a Dados	11
3.3.4 Comunicação em Tempo Real	12
3.3.5 Cálculo de distância de Percurso	12
4 Arquitetura	13
4.1 Arquitetura Geral do Sistema	14
4.2 Frontend	15

4.2.1	Biblioteca KMP para Mapa	15
4.2.2	Navegação e Funcionalidades	16
4.2.3	Componentes de Suporte à Camada de Apresentação	25
4.3	Backend	29
4.3.1	Modelo de Dados	29
4.3.2	Modelo ER	31
4.3.3	Rotas	33
4.3.4	<i>Load Balancing</i> Dinâmico e <i>Deployment</i>	38
4.3.5	Docker e Automação do Ambiente	38
5	Conclusões	39
	Referências	44

Lista de Figuras

4.1	Arquitetura do Projeto	13
4.2	Mapa de navegação entre ecrãs	16
4.3	Ecra de autenticação	17
4.4	Ecra principal	19
4.5	Ecra de definições (1)	21
4.6	Ecra de definições (2)	22
4.7	Ecra de perfil	23
4.8	Ecra de trilhos	24
4.9	UML do Modelo de Dados desenvolvido	29
4.10	UML do Modelo ER desenvolvido	31

Lista de Acrônimos

- API** Interface de Programação de Aplicações (Application Programming Interface). 7
- CIO** Coroutine I/O. 7
- DSL** Linguagem Especificada de Domínio (Domain Specific Language). 11
- ER** Entidade-Relação (Entity-Relationship). vii, x, xi, 2, 31, 32
- GPS** Sistema de Posicionamento Global (Global Positioning System). 12
- GPU** Unidade de Processamento Gráfico (Graphics Processing Unit). 15
- HTML** HyperText Markup Language. 15
- HTTP** Protocolo de Transferência de Hipertexto (Hypertext Transfer Protocol). 4, 7, 9, 10, 12, 26, 33
- JSON** JavaScript Object Notation. 33{37
- JVM** Máquina Virtual Java (Java Virtual Machine). 8, 12, 14, 38
- JWT** JSON Web Tokens. 9, 34{37
- KML** Keyhole Markup Language. 36
- KMP** Kotlin Multiplatform. iii, x, 2, 3, 5{7, 11, 14, 15, 39, 40
- LLM** Modelo de Linguagem em Grande Escala (Large Language Model). vii, 2, 39, 41
- MIME** Multipurpose Internet Mail Extensions. 36
- SHA** Algoritmo de Hash Seguro (Secure Hash Algorithm). 10
- SSE** Server-Sent Events. 12
- TCP** Protocolo de Controle de Transmissão (Transmission Control Protocol). 7
- UI** Interface de Utilizador (User Interface). 5, 15
- UML** Linguagem de Modelagem Unificada (Unified Modeling Language). xi, 29, 31
- WASM** WebAssembly. 15
- XML** Extensible Markup Language. 36

Capítulo 1

Introdução

Este capítulo tem como objetivo explicar ao leitor o problema encontrado, a importância deste, o que já existe como solução, qual será a solução a desenvolver pelo grupo e como está organizado o relatório.

1.1 Enquadramento

Durante os tempos modernos, os caminhantes e/ou corredores têm um problema principal relativamente aos trilhos/caminhos que pretendem percorrer: encontrar um que cumpra todas as suas preferências e expectativas.

Este paradigma cria diversas dificuldades e obstáculos, como perda de tempo em procura, planeamento pessoal excessivo, não registo de informações de estatística da caminhada/corrida, entre outros.

Visando este cenário, considerou-se pertinente o desenvolvimento de uma aplicação que resolva estes problemas e outros encontrados ao longo do seu desenvolvimento.

1.2 Estado da arte

Relativamente ao mercado desta área, existem diversas soluções (Strava e AllTrails) nas diferentes plataformas de aplicações, com funcionalidades notáveis e extensas, nomeadamente ao nível do registo de percursos e estatísticas de atividade. Ainda assim, considera-se que este projeto se distingue ao permitir que os utilizadores visualizem, em tempo real, a localização de outros participantes a percorrer o mesmo trilho, possibilitando, por exemplo, que um pai e um filho que correm a velocidades distintas se possam acompanhar mutuamente.

Apesar disto, considera-se que o principal objetivo do desenvolvimento desta aplicação assenta na exploração e estudo de novas ferramentas da área onde esta se encontra inserida, mais do que na diferenciação face às soluções já existentes no mercado.

1.3 Objetivos e metas

Este projeto visa o desenvolvimento de uma aplicação com extensão a multi-plataforma para caminhantes e corredores que disponibilize funcionalidades para a criação e caminhada de trilhos, guardando diversas estatísticas sobre estes, como distância total percorrida, número total de trilhos percorridos, entre outras.

Existem objetivos mais específicos como:

- Integração de um mapa interativo pelos utilizadores;
- Autenticação via OAuth/OpenID;
- Integração de um *chatbot* através de um LLM (Funcionalidade não implementada no final do projeto).

1.4 Resumo da solução proposta

Como solução ao problema descrito, desenvolveu-se então uma aplicação, principalmente para dispositivos Android, com KMP, a biblioteca MapBox e diversas outras tecnologias e ferramentas descritas ao longo deste relatório.

Esta aplicação visa ter as seguintes funcionalidades:

- Criação e autenticação de utilizadores através de plataforma Google ou email e password;
- Criação e caminhada de um trilho desenvolvido pela comunidade ou pelo próprio utilizador;
- Interação com um *chatbot* para ajuda na aplicação;
- Acesso a dados e estatísticas do utilizador.

1.5 Organização do relatório

Este relatório é composto pelos seguintes capítulos:

- Tecnologias e protocolos, (ver secção 2), que visa introduzir brevemente as tecnologias e protocolos utilizados, e as suas utilizações;
- Software, (ver secção 3), onde foi procurado explicar os dilemas e devidas escolhas sobre as tecnologias utilizadas;
- Arquitetura, (ver secção 4), onde é explicada a arquitetura geral do projeto, desde a sua conceção inicial até à sua implementação final, incluindo os Modelos de Domínio e ER, as rotas disponíveis e a infraestrutura de suporte;
- Conclusão, (ver secção 5), com o objetivo de explicar as dificuldades, funcionalidades não implementadas, as aprendizagens obtidas e como dar continuidade ao projeto.

Capítulo 2

Tecnologias e Protocolos

2.1 Tecnologias Adotadas

Foram escolhidas as seguintes tecnologias para o desenvolvimento desta aplicação:

- KMP[1] para o desenvolvimento da aplicação no seu todo, permitindo esta ser multi-plataforma;
- Ktor[2] para implementação do servidor de cliente;
- Exposed[3] para acesso à base de dados;
- R2DBC[4] para camada de transações do exposed;
- PostgreSQL[5] para gestão da base de dados;
- Redis[6] para comunicação de WebSockets entre servidores;
- Lettuce[7] para fazer a conexão entre o servidor e o Redis;
- MapBox Android[8] e MapBox GL JS[9] como bibliotecas de implementação de mapas;
- Compose Multiplatform[10] para interface de utilizador;
- Kermit[11] para registo de logs;
- BuildKonfg[12] para injeção de propriedades no BuildConfg de todos os alvos no KMP;
- Ktlint[13] para indentação automática do código;
- Maven Publish[14] para automação na publicação da biblioteca original de mapas para o Maven Central[15].
- OpenAPI Specifications (Swagger)[16] para documentação da API RESTful;
- nginx[17] para *proxy* e *load balancing*;
- Docker[18] para contentorização e escalabilidade horizontal dos serviços;
- Gradle[19] para compilação e gestão de dependências;
- ngrok[20] para exposição do ambiente local à Internet;

2.2 Protocolos

- HTTP para comunicação e tráfego de dados entre *Backend* e *Frontend*;
- JWT para o formato do token de acesso;
- Bearer para o esquema de transmissão de ambos os tokens (acesso e atualização) via cabeçalho *Authorization*;
- OAuth[21]/OpenID[22] para autenticação de utilizadores pela Google;
- WebSockets[23] para comunicação em tempo real entre servidor e cliente, em ambos os sentidos;

Capítulo 3

Software

3.1 Frontend

3.1.1 Tecnologia para Alvos

Ao iniciar o projeto, existiu a dúvida dos alvos a trabalhar. Houve a opção de apenas trabalhar sobre o alvo Android, ou englobar também a Web. Isto forçou uma escolha, seguir com Kotlin Multiplatform ou utilizar *Android Studio*[24] e *React*[25].

Os orientadores indicaram o KMP, ao qual, após devida procura, foi decidido prosseguir com esta tecnologia, devido ao facto de ser possível partilhar a UI e classes entre o *frontend* e o *backend*.

Esta escolha, recai na incerteza inicial sobre os alvos e a natureza do projeto, já que também existe o objetivo de manter a possibilidade para uma extensão a iOS.

Esta possibilidade denota-se lógica, sendo Android e iOS os 2 alvos mais interessantes para o projeto, tendo em conta que este se baseia muito na movimentação do utilizador. Apesar disso, a falta de equipamento para o devido desenvolvimento do alvo iOS, impossibilita a tentativa de trabalhar sobre este.

Esta decisão, acabou por levar a algumas consequências negativas no decorrer do projeto, devido há necessidade de utilizar bibliotecas KMP, que nem sempre existem, ou não são extensíveis a todos os alvos, por ainda ter uma comunidade pequena.

Devido a isso, no final do projeto, só foi possível desenvolver, até ao fim, o alvo Android, já que a Web, em KMP, ainda demonstra demasiadas debilidades, principalmente em projetos que dependem, em larga escala, de bibliotecas restritas (tal como os mapas) de terceiros.

3.1.2 Biblioteca de Mapa

Existiu uma decisao importante neste componente: qual biblioteca de mapas é que deve ser utilizada, sendo preferencialmente uma biblioteca destinada a KMP.

Foram avaliadas 2 bibliotecas, *OpenStreetMap*[26] e *Google Maps API* [27], mas ambas foram descartadas por nao serem KMP e necessitarem de informacoes bancárias.

Após uma pesquisa mais profunda, foram descobertas 2 bibliotecas, *MapLibre*[28] e *MapBox*. O *MapLibre* tinha mapa destinado a KMP e nao necessitava de nenhum tipo de autenticação, enquanto que o *MapBox* apenas tem bibliotecas separadas, *MapBox Android* e *MapBox GL JS*, necessita de autenticação e para emails que nao sejam universitários é necessário colocar dados de conta bancária. Importante denotar que *MapLibre* é uma bifurcação de *MapBox*.

Tendo isto em conta, foi decidido começar a implementação a partir do *MapLibre*.

Contudo, ao decorrer do projeto, foram descobertas falhas no código original da biblioteca e também foi descoberto que a biblioteca nao tinha uma implementação completa para a *Web*, cerca de 30% do desenvolvimento dado ao Android. Ficou por tomar uma nova decisao: continuar com o *MapLibre*, resolvendo os problemas encontrados e contribuir com essa resolução para a comunidade KMP, ou transitar para *MapBox*, ficando com a necessidade de trabalhar em implementações específicas para cada alvo.

Sobre esta decisao, e tendo em conta o tempo desperdiçado nos problemas do *MapLibre*, foi decidido transitar para o *MapBox*, sendo esta escolha considerada mais segura.

3.2 Ktor

O uso do *Ktor*, como tecnologia central de comunicação, tanto no *Frontend* como no *Backend*, surgiu como uma consequência natural e estratégica, devido à utilização do KMP como tecnologia central do projeto.

3.2.1 Ktor Client

No *Frontend* (alvo Android), o *Ktor Client* foi utilizado para toda a comunicação HTTP com o servidor, incluindo a recepção de localização em tempo real via *WebSocket*. Para este alvo, foi escolhido o motor *OkHttp*, em detrimento do motor *CIO* (*Coroutine I/O*), por oferecer uma integração mais madura no ecossistema Android, nomeadamente na estabilidade perante mudanças de rede e execução em segundo plano, características essenciais para uma ligação *WebSocket* contínua. Adicionalmente, o *OkHttp* suporta HTTP/2, permitindo multiplexar vários pedidos HTTP sobre a mesma ligação TCP, reduzindo a necessidade de estabelecer múltiplas ligações independentes e melhorando a eficiência da comunicação.

No *Backend*, o *Ktor Client* também é utilizado, mas apenas para comunicação pontual com serviços externos (nomeadamente a API de *OAuth* da Google). Por se tratar de pedidos HTTP de curta duração por autenticação, sem um volume de pedidos que justifique a multiplexagem do HTTP/2 (não suportado pelo motor *CIO*), foi escolhido o motor *CIO*, mais leve e sem dependências adicionais.

3.2.2 Ktor Server

Consequentemente, surgiu o dilema arquitetural para o *Backend*: manter o *Ktor*, ou adotar o *Spring Boot*[29], uma das *frameworks* mais utilizadas para desenvolvimento de aplicações *backend* na JVM.

No momento da decisão, ainda não estava definido se o projeto incluiria um alvo Web (Kotlin/JS), logo não seria desejável depender da serialização baseada em reflexão fornecida por defeito pelo *Spring Boot* (*Jackson*). Caso o projeto viesse a incluir alvo Web, seria necessário recorrer a *kotlinx.serialization*, biblioteca multiplataforma utilizada pelos módulos partilhados. O *Ktor* oferece integração nativa com esta biblioteca através do *ContentNegotiation*, evitando a coexistência de mecanismos distintos de serialização, configuração adicional que teria de ser feita utilizando *Spring Boot*.

Outro fator é que o *Spring Boot*[29] é um *framework* "pesado" e altamente opinativo, que traz consigo um ecossistema vasto de dependências (injeção de dependências, anotações complexas, reflexão). O *Ktor* segue uma filosofia *micro-framework* e minimalista baseada em *plugins*. O servidor arranca apenas com o estritamente necessário, resultando num tempo de inicialização muito mais rápido e menor consumo de memória.

Relativamente ao modelo de concorrência, tanto o *Spring Boot* (através do *Spring WebFlux*) como o *Ktor* suportam funções *suspend* em corrotinas Kotlin. O *Spring MVC* (a variante tradicional do *Spring Boot*) mantém um modelo bloqueante de *thread-per-request*. Embora o Spring ofereça a variante *Spring WebFlux* para colmatar esta falha, o *Ktor* já é baseado sobre um modelo não-bloqueante (corrotinas do Kotlin). Isto permite escalar milhares de ligações simultâneas utilizando uma *pool* reduzida de *threads*, adequando-se perfeitamente à natureza de comunicação em tempo real exigida pela aplicação.

Como motor de servidor, foi escolhido o *Netty*, por ser o motor recomendado pelo *Ktor* para ambientes de produção, com suporte a comunicação assíncrona.

3.3 Backend

3.3.1 Segurança

Apesar de não serem manipulados dados sensíveis dos clientes, como por exemplo pagamentos, ainda é necessário ter atenção à segurança dos dados, já que, muitas pessoas utilizam sempre a mesma palavra-passe para o email e é necessário minimizar os riscos de fuga de informação.

3.3.1.1 Autenticação

É importante que o utilizador possa comprovar que realmente é o dono da conta e nesse sentido foram desenvolvidos 2 métodos de autenticação.

O utilizador pode escolher autenticar-se por *OAuth* da Google, ou por `\email` e `\password`. Após isso utiliza-se o método de tokens de `\access` e `\refresh`.

Para poder-se autenticar um cliente Android, foi necessário modificar o mecanismo de resposta normal, enviando um *deep link*, que o *browser* do Android irá repassar à aplicação, por via de *intent*.

Este link, tem 2 tipos de resposta: ou transmite um parâmetro de *query* `\error`, ou um conjunto de parâmetros, `\code`, `\email`, `\username` e `\new`.

O `\code`, tem a mesma criação de um `\refresh` token, mas com uma vida útil de apenas 2 minutos. O `\username`, pode ser tanto o nome de utilizador do email utilizado no *OAuth*, ou um nome de 32 caracteres construído de forma aleatória (podendo conter letras maiúsculas, minúsculas, `\_` ou `\.`). O `\new` apenas indica se o *OAuth* foi criação de conta ou apenas *login*.

O token de acesso é um JWT, que guarda o nome do utilizador e tem uma vida útil de 15 minutos. O token `\refresh` é o token de longa duração utilizado para recriar o token de acesso, sendo utilizado apenas quando o token de acesso expira, ou em métodos importantes como eliminação e alteração de dados da conta. O tempo de vida útil é de 90 dias, mas como manter um token durante tanto tempo, e com múltiplo usos, pode ser arriscado, o `\refresh` token só pode ser utilizado uma vez, recriando sempre um novo.

Este token é gerado utilizando o método *SecureRandom.getInstanceStrong()* da biblioteca `\java.security` e tem tamanho de 32 bytes. Ambos são transmitidos ao servidor a partir do cabeçalho *Authorization*, passado nos pedidos HTTP, sempre com o esquema *Bearer*.

```
ByteArray(TOKEN_BYTES).let { byteArray ->
    SecureRandom.getInstanceStrong().nextBytes(byteArray)
    getUrLEncoder().encodeToString(byteArray)
}
```

3.3.1.2 *Hashing*

O *hashing* é tão importante quanto a autenticação, pois caso alguém tenha acesso aos dados da nossa base de dados, caso os valores importantes estejam guardados sem *hashing*, o invasor terá acesso a todas as contas do sistema.

Para suprir esta necessidade, foram escolhidos 2 métodos de *hashing*, BCrypt e SHA256. O motivo para o qual ficou decidido utilizar 2 métodos e não apenas 1, tem a ver com a necessidade de *hashing*, já que o BCrypt faz *hash* às palavras-chave e o SHA256 faz *hash* aos tokens. Mesmo que o token já tenha um forte *hashing*, devido ao *SecureRandom*, colocar o valor diretamente na base de dados ainda é acompanhado de riscos, logo decidiu-se utilizar este método de *hashing*.

Por outro lado, a palavra-chave sendo uma informação mais importante, pelos motivos indicados na seção 3.3.1, deve ter um método de *hashing* mais forte, o BCrypt, vindo da biblioteca de segurança do sistema *Spring*, (*springframework.security*[30]).

3.3.2 Tratamento de Erros

Para o tratamento de erros, foi tomada uma abordagem de unir os tipos de erros dentro de *sealed class* para cada serviço disponível (neste caso, *User*, *Trail* e *Hike*), sendo estas *sealed classes* extensões de uma outra *sealed class* geral chamada *\DomainError*", que contém possíveis erros que possam acontecer ainda no tratamento dos pedidos HTTP.

Para encapsular os erros utilizamos uma *sealed class* *\Either*", tendo o método *\failure*" e o método *\success*", sendo *\failure*" do tipo uma das classes de erro e *\success*" o resultado desejado ao pedido exercido. Esta classe é uma imitação da classe *Result* do *Kotlin*, mas como só tivemos conhecimento da sua existência já numa fase mais avançada do projeto, decidimos seguir com a classe que já tínhamos trabalhado.

Comparativamente a outras possíveis implementações, como lançar exceções, a nossa decisão pode apresentar menor overhead, evitando os custos associados ao lançamento de exceções.

Este *overhead* advém da necessidade de criação de um stack trace, coisa que não acontece com a classe *\Either*". Outras vantagens são a nível computacional, como melhor previsibilidade dos erros e menor necessidade de compreensão das possíveis exceções lançadas pelo método utilizado.

3.3.3 Acesso a Dados

A tecnologia que escolhemos para a camada de acesso a dados foi o *Exposed*. Esta foi-nos indicada pelos orientadores, como uma solução interessante para um projeto KMP. Existiu a necessidade de decidir entre escolher tecnologias como *JDBC* [31] ou *JDBI* [32] e *Exposed*, sendo a decisão final unânime o *Exposed*.

Uma avaliação mais aprofundada revelou as vantagens fornecidas por esta tecnologia. Um dos principais motivos da escolha é a API fortemente tipificada, que fornece verificação de erros em tempo de compilação, com base numa DSL, ao contrário das outras opções, que só demonstram os erros em tempo de execução e necessitam de escrita de consultas em formato textual.

Outro motivo bastante relevante nesta escolha foi o uso do *Ktor* como servidor. O facto do *Ktor* ser um servidor baseado em corrotinas, indica que este é um servidor não bloqueante. Isto faz com que a utilização de tecnologias bloqueantes no acesso a dados não retire o melhor proveito sobre este servidor.

Por isso, foi escolhida a utilização do *Exposed*, tecnologia que pode tanto ser bloqueante como não bloqueante, dependendo do padrão de conectividade escolhido. O *Exposed* funciona com 2 padrões, *jdbc* e *r2dbc*, baseados em *threads* e corrotinas, respetivamente.

Isto indica que, para utilização de acesso a dados não bloqueante, a escolha ideal de conectividade, para aliar ao *Ktor* não bloqueante, recai sobre o *r2dbc*, já que é totalmente baseado em corrotinas.

Apesar de ser a escolha ideal, necessita de uma atenção especial, tendo em conta que existe um número máximo de conexões ativas no *PostgreSQL*, sendo necessário utilizar uma biblioteca adjacente do *r2dbc*. A biblioteca em questão, trata-se da `\io.r2dbc:r2dbc-pool`, que serve para limitar a quantidade de conexões criadas, pois este padrão não tem um limite máximo pré-definido na criação de corrotinas, ao contrário do *jdbc* com as *threads*.

Se não for utilizada uma *pool* que limite o número de *coroutines* ativas, existe a possibilidade do número máximo de conexões ser alcançado, o que fará com que novas conexões sejam recusadas pelo *PostgreSQL*. Isto levaria a potenciais falhas em cascata na aplicação.

3.3.4 Comunicaçao em Tempo Real

Esta é uma componente importante do projeto, pois foi projetado que os utilizadores possam percorrer caminhos, observando a posição de outros utilizadores no mesmo percurso. Para resolver esta questão, é necessário que os utilizadores disponibilizem as suas localizações ao servidor. Tendo em conta que é pedida a localização exata do utilizador no Android, e esta localização é atualizada num espaço muito curto, serão utilizados *WebSockets*.

Os *WebSockets* são mais vantajosos que os *SSE*, já que são *full-duplex* (bidirecionais), enquanto os *SSE*, são *simplex* (unidirecionais).

Como a localização será enviada de forma rápida, avaliámos que manter a ligação aberta com *WebSocket*, é menos custoso do que fazer um pedido HTTP a cada nova localização.

Para apoiar os *WebSockets*, foi projetada a utilização de *Redis Pub/Sub*, com o objetivo de ter uma comunicação entre *WebSockets* alojados nas diferentes JVMs, já que ficou decidido a implementação de *load balancing* dinâmico à base de *nginx*.

3.3.5 Calculo de distância de Percurso

Tendo em conta que precisamos saber a distância de um percurso, tivemos de escolher um método matemático que pudesse ser usado para fazer este calculo.

Existem então 2 opções, distância euclidiana e distância de haversine; destas, foi escolhida a distância de haversine. Isto porque a distância euclidiana calcula a distância em linha reta entre dois pontos, o que para pontos distantes poderia causar problemas, enquanto que a distância de haversine leva em conta a curvatura da Terra.

É relativamente óbvio que ambos os métodos funcionam bem para distância curtas, levando vantagem a distância euclidiana. Contudo, como é possível existirem pontos de um percurso com distância largas, foi decidido optar pelo método mais preciso, mesmo perdendo velocidade de computação.

Também vale ressaltar que a distância de haversine normalmente é um dos métodos mais utilizados para calcular distância com coordenadas GPS, sendo utilizado pelo *Google Maps* em certos casos.

Capítulo 4

Arquitetura

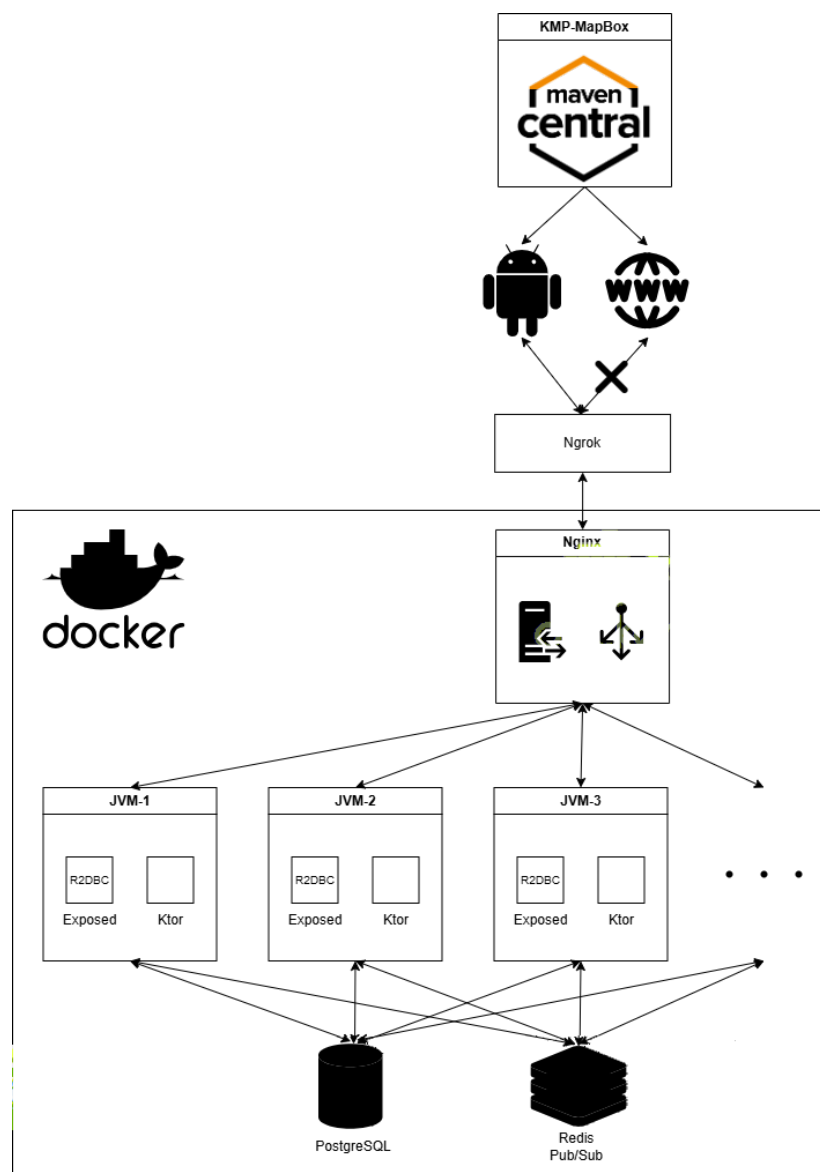


Figura 4.1: Arquitetura do Projeto

4.1 Arquitetura Geral do Sistema

A figura 4.1 apresenta a arquitetura global inicialmente prevista para o projeto. Embora a arquitetura tenha sido concebida para suportar clientes Android e Web através de Kotlin Multiplatform, apenas o cliente Android foi concluído. A implementação do cliente Web foi iniciada, mas não foi finalizada devido a problemas de compatibilidade encontrados em bibliotecas KMP com o alvo Web, problemas esses que não foram possíveis de resolver dentro do tempo disponível para o projeto. Os alvos recorrem à biblioteca KMP-MapBox, desenvolvida pelo grupo, cuja sua necessidade é explicada na seção 4.2.1.

Para expor os servidores à Internet, um requisito necessário para a comunicação com a aplicação Android, é utilizado o *ngrok*, com toda a comunicação externa encaminhada através de um servidor *nginx*, seção 4.3.4. Este servidor atua como *Reverse Proxy* e é responsável pelo *load balancing* dinâmico, distribuindo os pedidos equitativamente pelas várias JVMs (instâncias da aplicação Ktor) executadas em contentores Docker, seção 4.3.5. Esta abordagem permite que a aplicação seja escalável horizontalmente.

No que diz respeito à camada de dados, cada instância da aplicação comunica com uma base de dados PostgreSQL através da *framework Exposed* sobre *r2dbc*, seção 3.3.3.

Adicionalmente, o sistema utiliza Redis para a sincronização de eventos em tempo real (*WebSockets*) entre os servidores. Como as ligações *WebSocket* mantêm estado e ficam restritas a uma JVM específica, o Redis atua como um intermediário de comunicação. No contexto do projeto, isto é utilizado para sincronizar a localização de cada utilizador num determinado trilho, pois se um utilizador conectado à JVM-1 atualizar a sua localização, o Redis encarrega-se de propagar esse evento para as restantes instâncias (como a JVM-2). Desta forma, garante-se que todos os participantes no trilho recebem a atualização instantaneamente, independentemente do servidor ao qual estejam ligados, viabilizando assim a escalabilidade horizontal da arquitetura.

4.2 Frontend

4.2.1 Biblioteca KMP para Mapa

Foi decidido criar uma biblioteca KMP, utilizando as bibliotecas *MapBox Android* e *MapBox GL JS*.

Tomou-se esta decisão, pois, tendo em conta que já era necessário implementar o UI separado para estas 2 bibliotecas, considerou-se que criar uma biblioteca KMP e publicá-la no *Maven Central* seria uma forma de demonstrar a contribuição do grupo para a comunidade KMP.

Esta biblioteca visa implementar mapa para Android e Web da forma mais abstrata possível, mas, como referido na secção 3.1.1 encontraram-se algumas adversidades.

Como o *MapBox GL JS* não está feito para WASM, logo trabalha sobre HTML, teve de se implementar em *jsMain*, mas ao decorrer do desenvolvimento da biblioteca, foram descobertos outros problemas. Começou-se por tentar utilizar a API *ComposeUI* para *JavaScript*, mas ainda é experimental e está inacabada por parte da JetBrains. Isso fez com que, quando se tentou incluir *overlays* com o *ComposeUI* utilizado no Android, não era possível visualizá-los.

Este problema ocorre porque o *ComposeUI* constrói um "Canvas" representado como um único elemento (que é um *WebGL*), e o *MapBox GL JS* constrói o seu próprio *WebGL*. Acabou por ocorrer uma concorrência pelos recursos de GPU onde não seria possível interligar os 2 *WebGL*.

Apesar de tudo, foi possível conciliar o uso de *ComposeUI* com HTML, devido à classe *HTMLElement*, que aliado à função *HtmlElementView*, permite utilizar elementos HTML dentro de *ComposeUI*. Contudo, esta solução não é a melhor, pois continua a não permitir o uso de *Composable*, em conjunto com o *WebGL* do mapa. A necessidade de não utilizar *ComposeUI*, faz com que seja impossível unir os mapas numa única função, o que seria desejável.

Para a biblioteca ficar otimizada, a implementação do *MapBox iOS* daria um bom incremento, mas o essencial seria o *MapBox* fornecer um mapa WASM, ou encontrar outra biblioteca de mapas seja extensível a WASM.

4.2.2 Navegação e Funcionalidades

A aplicação é constituída por 7 ecras principais nos quais o utilizador pode realizar diversas ações e ter disponível diversas informações. A figura seguinte 4.2 descreve um possível mapa de navegação da aplicação e como o utilizador pode navegar entre os ecras disponíveis.

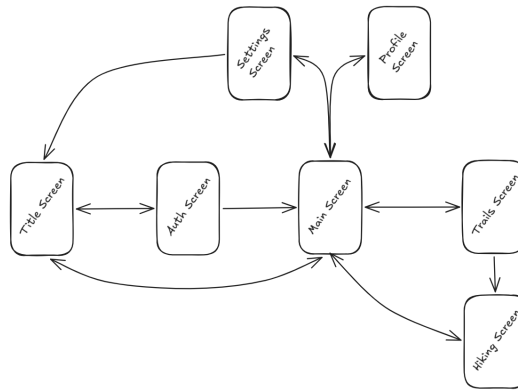


Figura 4.2: Mapa de navegação entre ecras

4.2.2.1 Ecrã de Início (ou *Title Screen*)

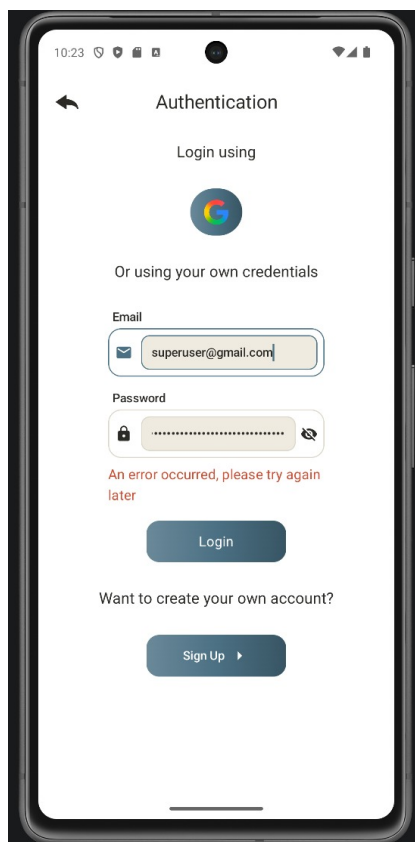
O ecrã de início é o primeiro ecrã apresentado ao utilizador quando este entra na aplicação. As suas funcionalidades são as seguintes:

- Apresentar ao utilizador informações relevantes sobre a aplicação e uma breve descrição desta;
- Redirecionar o utilizador para o ecrã de autenticação caso este carregue no botão correspondente;
- No caso do utilizador se encontrar autenticado por causa de uma sessão anterior, redireciona este para o ecrã principal diretamente sem precisar de passar pelo ecrã de autenticação após uma animação que confirma o sucesso desta autenticação e o respetivo nome do utilizador.

4.2.2.2 Ecrã de autenticação (ou *Auth Screen*)

O ecrã de autenticação serve para o utilizador se autenticar, como pode ser visto no conjunto de figuras 4.3. Tem as seguintes funcionalidades:

- Iniciar sessão através de conta Google ou com credenciais específicas de utilizador, sendo estas email e password 4.3a;
- Registar um utilizador através de uma conta Google ou com credenciais específicas de utilizador, sendo estas nome, email, password e confirmação desta 4.3b;
- No caso de o utilizador iniciar sessão pela primeira vez ou criar conta com o Google, irá ser disponibilizado um diálogo onde o utilizador pode ver o nome atribuído e pode assim modificar este ou então adicionar uma password à conta para utilizar o outro método de autenticação;
- Pode voltar para o ecrã inicial através de um deslize para trás no ecrã ou pressionando a seta para trás no topo do ecrã;
- No caso de o utilizador se autenticar corretamente, é demonstrada uma animação a confirmar este sucesso, redirecionando assim o utilizador para o ecrã principal.



(a) Iniciar Sessão

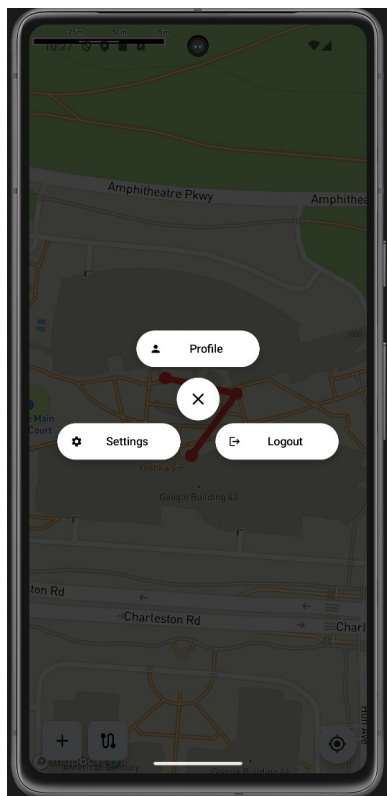
(b) Criar Conta

Figura 4.3: Ecrã de autenticação

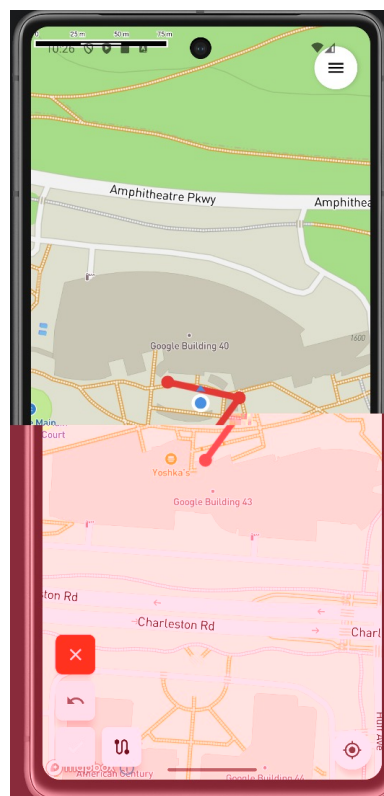
4.2.2.3 Ecrã Principal (ou *Main Screen*)

O ecrã principal permite ao utilizador ver a sua localização e utilizar diversas funcionalidades, como se pode observar pelo conjunto de figuras 4.4. Estas são:

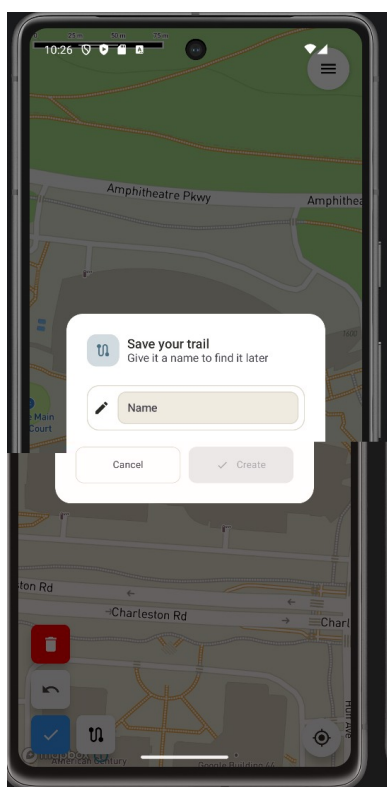
- No topo do ecrã, carregar no **Ícone de Menu** para abrir um menu no meio da tela do dispositivo em que o utilizador pode seleccionar entre navegar para o ecrã de perfil ou ecrã de definições ou ainda terminar sessão e voltar para o ecrã inicial da aplicação 4.4a;
- No canto inferior esquerdo, carregar no **Ícone "+"** disponibiliza outros três ícones 4.4b:
 - **Ícone de Certo** - confirma a criação de um trilho após o utilizador carregar no mapa para marcar os diversos pontos do trilho;
 - **Ícone de Retrocesso** - desfaz o último ponto confirmado pelo utilizador para a criação do trilho;
 - **Ícone "x"** - Cancelar a criação do trilho ou fechar este mini menu.
- Caso o utilizador confirme a criação do trilho, aparece um diálogo para que o utilizador possa inserir o nome do trilho a ser criado 4.4c;
- Ainda no canto inferior esquerdo, carregar ainda no ícone de trilho redireciona o utilizador para o ecrã de trilhos;
- No canto inferior direito, carregar no ícone de localização centra o mapa na localização do utilizador;
- No mapa em si disponibilizado no ecrã, caso haja trilhos, são disponibilizados no próprio mapa para o utilizador ver e iniciar uma caminhada nestes caso queira 4.4d.



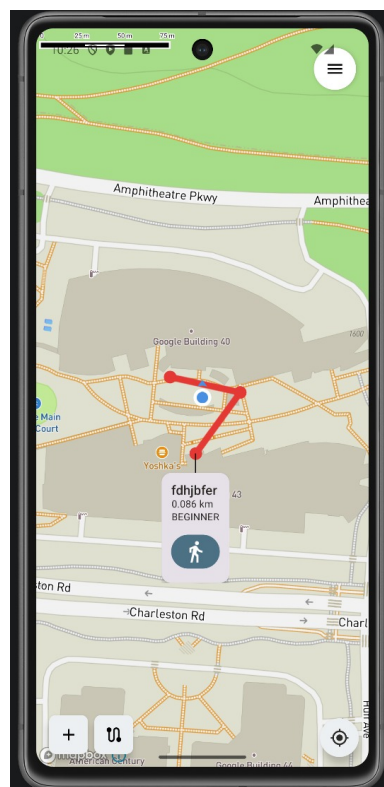
(a) Menu



(b) Criação de trilho



(c) Diálogo de criação de trilho



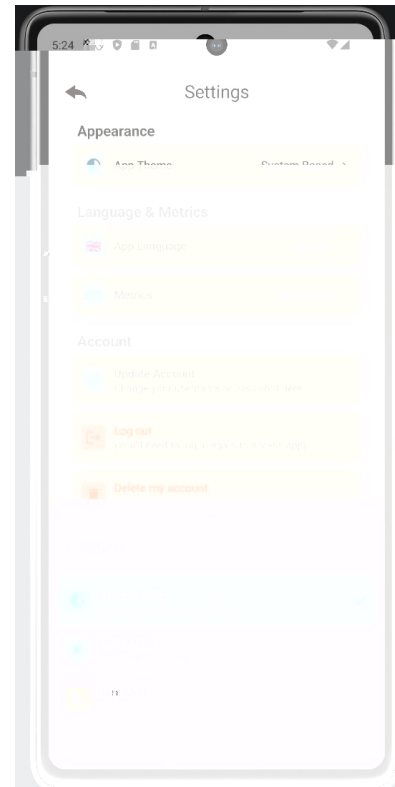
(d) *Display* de um trilho

Figura 4.4: Ecra principal

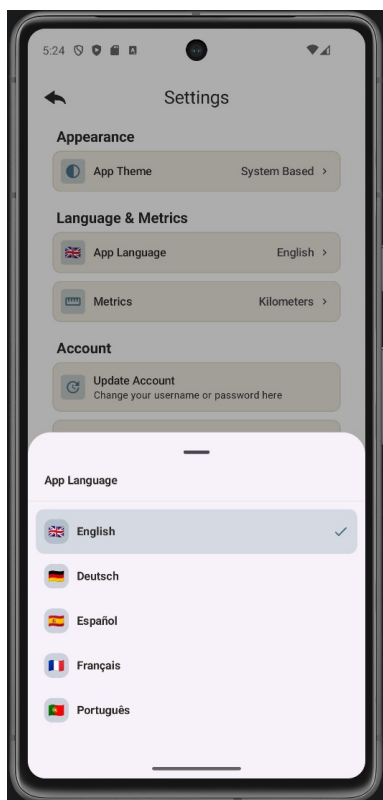
4.2.2.4 Ecrã de Definições (ou *Settings Screen*)

O ecrã de definições serve para o utilizador mudar diversas funcionalidades e aspetos da aplicação, como se pode observar nos conjuntos de figuras 4.5 e 4.6. O utilizador tem disponível as seguintes funcionalidades:

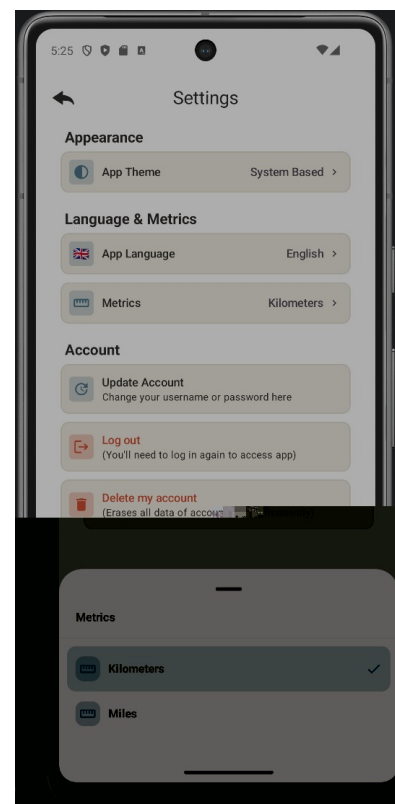
- Na secção de **Aparência**, carregar em **Tema da Aplicação** abre um menu onde o utilizador pode escolher entre manter o tema consoante o que está definido no dispositivo, tema claro ou tema escuro. Este é alterado numa animação caso o utilizador decida mudar o tema 4.5b;
- Na secção de **Idioma e Métricas**:
 - **Idioma da Aplicação** - abre um menu onde o utilizador pode escolher entre Inglês, Português, Alemão, Francês e Espanhol. Caso o utilizador decida trocar o idioma, é apresentado um diálogo de confirmação a avisar que a aplicação irá reiniciar para repor o idioma 4.5c;
 - **Métricas** - abre um menu onde o utilizador pode escolher entre quilómetros e milhas para serem vistos na aplicação inteira como métrica 4.5d.
- Na secção de **Conta**:
 - **Atualizar Conta** - abre um diálogo onde o utilizador consegue alterar o seu nome de utilizador ou password. No caso do utilizador alterar estes corretamente, é realizada uma animação a confirmar a atualização com sucesso 4.6a;
 - **Terminar Sessão** - abre um diálogo para o utilizador confirmar que pretende terminar a sua sessão, avisando que terá de iniciar novamente caso deseje aceder à aplicação e à sua conta. No caso de sucesso, o utilizador é redirecionado para o ecrã inicial 4.6b;
 - **Apagar a minha conta** - abre um diálogo onde o utilizador é avisado que, no caso de completa esta ação, os seus dados e conta serão apagados permanentemente e, como método de segurança e precaução, este terá de digitar uma certa frase para que possa confirmar a eliminação da conta 4.6c.
- Pode voltar para o ecrã principal através de um deslize para trás no ecrã ou pressionando a seta para trás no topo do ecrã.



(b) Escolha do tema

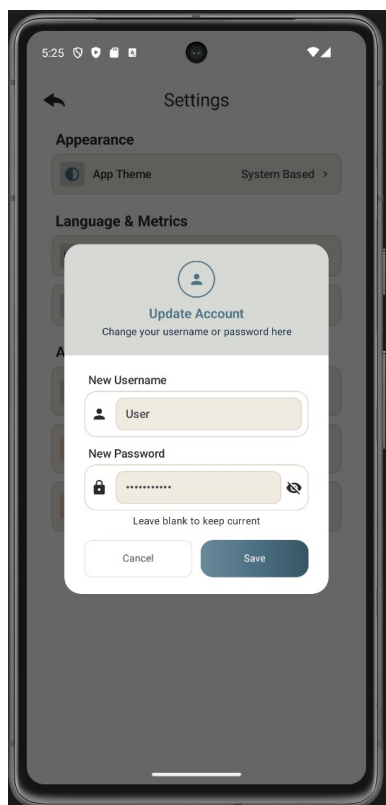


(c) Escolha da linguagem

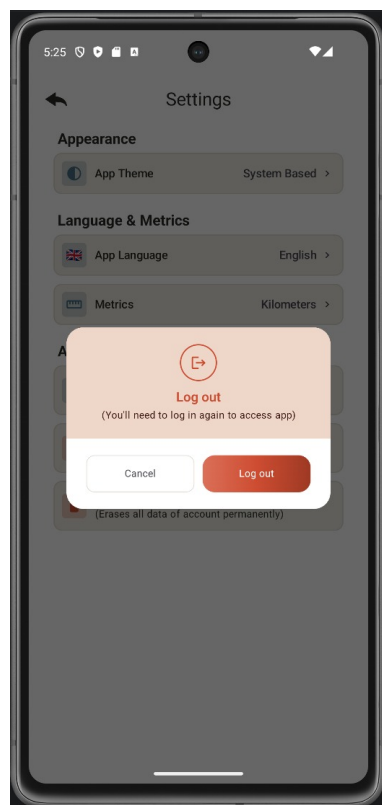


(d) Escolha da métrica

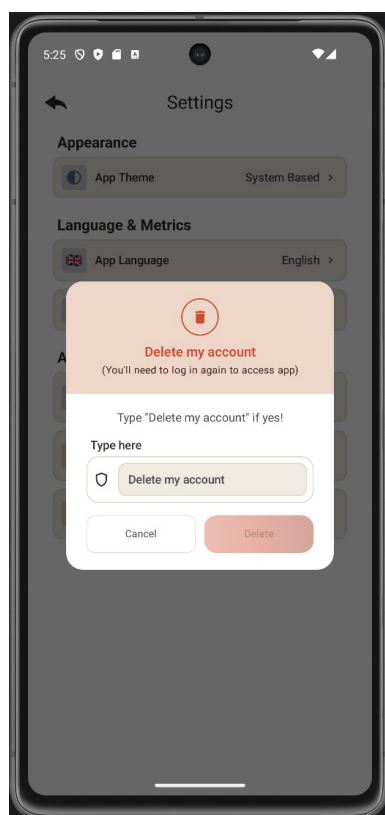
Figura 4.5: Ecrã de definições (1)



(a) Atualização do utilizador



(b) Terminar Sessão



(c) Apagar a conta do utilizador

Figura 4.6: Ecrã de definições (2)

4.2.2.5 Ecrã de Perfil (ou *Profile Screen*)

O ecrã de perfil tem como propósito dispor diversas informações ao utilizador, como se pode observar na figura 4.7. Estas informações são as seguintes:

- O seu nome de utilizador e rank na aplicação;
- As suas estatísticas, sendo estas total de trilhos completos, total de distância caminhada e tempo total de caminhada.

O utilizador pode voltar para o ecrã principal através de um deslize para trás no ecrã ou pressionando a seta para trás no topo do ecrã.

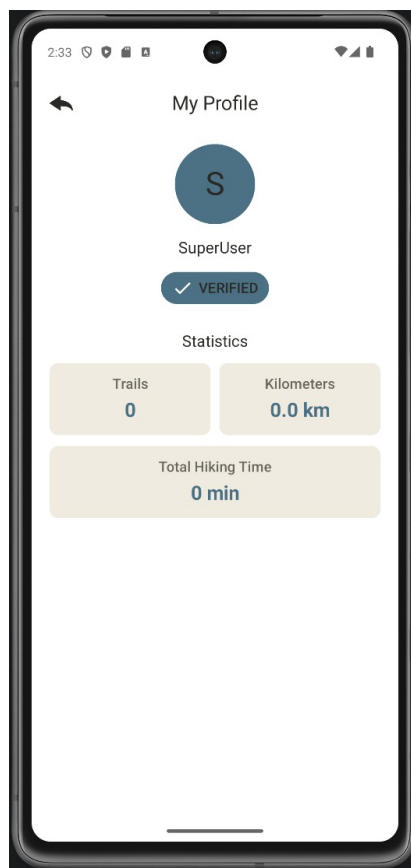


Figura 4.7: Ecrã de perfil

4.2.2.6 Ecrã de Trilhos (ou *Trails Screen*)

O ecrã de trilhos fornece aos utilizadores os trilhos disponíveis, como na figura 4.8. Tem as seguintes funcionalidades:

- Através da barra de pesquisa no topo do ecrã, pesquisar por nome do trilho irá filtrar os trilhos e dispor ao utilizador consoante o nome inserido;
- Ainda no topo, é possível carregar nos botões de Base e Pessoal para filtrar os trilhos consoante esta preferência;
- Ao carregar num trilho disponível, passa a ser possível ao utilizador iniciar uma caminhada neste;
- Pode voltar para o ecrã principal através de um deslize para trás no ecrã ou pressionando a seta para trás no topo do ecrã.

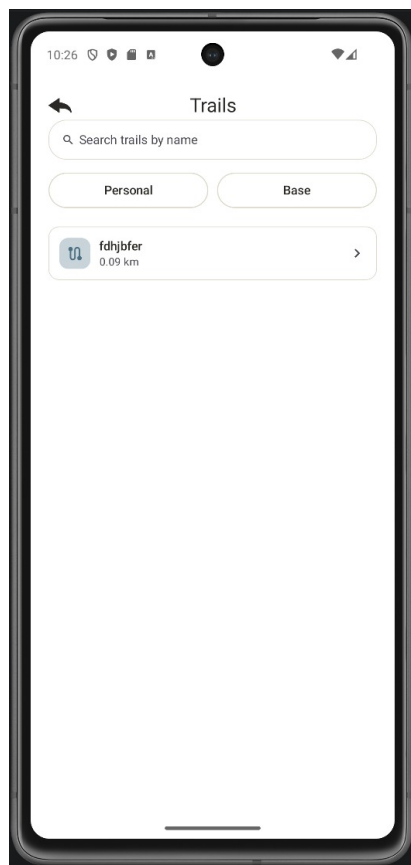


Figura 4.8: Ecrã de trilhos

4.2.2.7 Ecrã de Caminhada (ou *Hiking Screen*)

O ecrã de caminhada serve para o utilizador caminhar o trilho que selecionou. Visualmente quase idêntico ao ecrã principal, secção 4.2.2.3.

Ao invés de botões de criação, pesquisa de trilhos e menu, são fornecidos 2 botões, um para cancelar a caminhada e outro para finaliza-la, o último só ficando disponível no momento em que o utilizador cumpre os requisitos, precisando aproximar-se a 10 metros do ponto final.

No mapa em si, só aparecerá 1 trilho, sendo o trilho onde se caminha, as localizações dos utilizadores a percorrer o trilho naquele instante e o botão para seguir a sua localização.

4.2.3 Componentes de Suporte a Camada de Apresentação

Além dos ecrãs apresentados anteriormente, a aplicação tem um conjunto de componentes responsáveis por gerir o estado, a lógica de negócio, o acesso a dados e a navegação.

4.2.3.1 Repositórios

Os repositórios são os componentes responsáveis pela persistência local de dados. Foram implementadas duas interfaces:

- **UserRepository** - responsável por guardar e obter os tokens de sessão do utilizador (acesso e atualização, juntamente com o seu instante de expiração), bem como os seus detalhes básicos (identificador, nome de utilizador e *rank*). Disponibiliza ainda um método para limpar esta informação, utilizado sempre que a sessão do utilizador termina ou expira;
- **SettingsRepository** - responsável por guardar as preferências do utilizador relativas a tema da aplicação, idioma e métricas. Ao contrário do UserRepository, esta interface é implementada através de *expect/actual*, uma vez que o mecanismo de armazenamento local nativo difere consoante a plataforma.

4.2.3.2 Serviços

Os serviços fazem a comunicação com o servidor, sendo responsáveis por executar as ações pedidas pelo utilizador em pedidos HTTP (ou *WebSocket*, no caso da caminhada em tempo real) e por processar as respetivas respostas.

Cada domínio da aplicação (utilizadores, trilhos e caminhadas) possui uma interface de serviço (*UserService*, *TrailService* e *HikeService*) e uma respetiva implementação HTTP (*UserHttpService*, *TrailHttpService* e *HikeHttpService*).

Todas as implementações HTTP estendem uma classe abstrata comum, *Service*, que tem funcionalidades gerais, nomeadamente:

- Verificação da validade da sessão local antes de qualquer pedido, evitando chamadas desnecessárias ao servidor caso o utilizador já não esteja autenticado;
- Renovação automática do *token* sempre que uma resposta indique a sua expiração;
- Tratamento de respostas de erro, convertendo-as num tipo `Either<String, T>` que representa, de forma explícita, o sucesso ou falha de cada operação;
- Suporte à criação de *streams* via *WebSocket*, incluindo lógica de reconexão em caso de expiração do *token* durante o estabelecimento da ligação, utilizado no acompanhamento de caminhadas em tempo real.

Esta centralização evitou a duplicação de lógica de autenticação e tratamento de erros em cada um dos serviços concretos, e o tipo de retorno `Either` garante que, em qualquer chamada a um serviço, o consumidor é obrigado a tratar explicitamente ambos os casos de sucesso e falha.

4.2.3.3 ViewModels e States

Cada ecrã da aplicação possui um ViewModel associado (por exemplo, AuthViewModel, MapViewModel ou HikingViewModel), responsável por gerir o seu estado e fazer as respetivas chamadas aos serviços necessários.

O estado de cada ecrã é modelado através de uma interface State própria (por exemplo, AuthState ou TrailFetchState), com variantes que representam os diferentes momentos de uma chamada a um serviço: Idle, Loading, Success e Error (contém uma mensagem de erro). Este estado é exposto pelo ViewModel através de um StateFlow, permitindo que a interface gráfica observe e reaja automaticamente a alterações, sem necessidade de gestão manual de estado por parte dos ecrãs.

O fluxo típico de uma operação é o seguinte: o ecrã invoca uma função do ViewModel (por exemplo, login); o ViewModel atualiza o estado para Loading e invoca o serviço correspondente dentro do seu *scope* de corrotinas (*viewModelScope*); consoante o resultado devolvido pelo serviço (um *Either*), o estado é atualizado para Success ou Error.

A criação de instâncias de ViewModels é feita através de *factories* (*viewModelFactory* { *initializer* { ... } }), que permitem injetar manualmente as dependências necessárias (Serviços e Repositórios) sem recorrer a uma biblioteca de injeção de dependências externa.

4.2.3.4 NavigationProvider

A navegação entre ecras é gerida através de um componente próprio, o `NavigationProvider`, construído sobre a biblioteca *Navigation 3* e o seu modelo de *backstack*.

Cada ecrã da aplicação corresponde a uma variante de uma interface, `Route`, que implementa `NavKey` e é serializável, permitindo que o *backstack* seja persistido. Algumas rotas transportam ainda parâmetros próprios, como é o caso de `Auth` (que pode incluir um nome de utilizador, uma indicação de novo registo ou uma mensagem de erro) ou `Hike` (que transporta o trilha selecionado).

A função `navigationEntryProvider` é responsável por mapear cada `Route` para um `NavEntry`, sendo invocada automaticamente pelo `NavDisplay` sempre que uma nova rota é adicionada ao *backstack*. Para cada rota, esta função:

- Instancia o(s) `ViewModel(s)` necessário(s) para o ecrã correspondente, através das respetivas *factories*, injetando os serviços e repositórios recebidos como parâmetro;
- Constrói e apresenta o *composable* do ecrã, passando-lhe o(s) `ViewModel(s)` criado(s) e as funções de navegação (*callbacks*) necessárias para transitar para outros ecras.

Esta abordagem permite que a lógica de navegação esteja centralizada e desacoplada dos próprios ecras, que apenas invocam *callbacks* (por exemplo, `onBack` ou `onHike`) sem conhecimento direto do mecanismo de navegação subjacente. Adicionalmente, a utilização de `rememberViewModelStoreNavEntryDecorator` garante que os `ViewModels` de um ecrã são corretamente destruídos quando este é removido do *backstack*, evitando fugas de memória.

4.3 Backend

4.3.1 Modelo de Dados

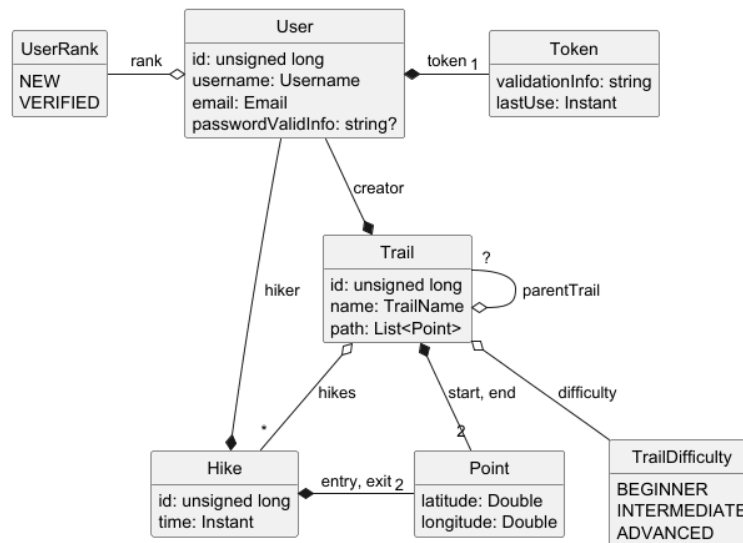


Figura 4.9: UML do Modelo de Dados desenvolvido

O modelo de dados, figura 4.9, ilustra a arquitetura central do sistema de gestão de trilhos e registo de caminhadas do projeto. O modelo assenta em 3 classes principais: User (o interveniente), Trail (o percurso) e Hike (a atividade).

4.3.1.1 User

A classe User, representação do cliente que utilizará a aplicação, contém alguns atributos de identificação como `\id`, `\username`, `\email` e `\passwordValidInfo`. É importante notar que `passwordValidInfo` pode ser nulo, isto porque projetamos 2 métodos de um utilizador se identificar e autenticar, por `\email` e `\password`, ou por *OAuth* da Google e neste caso, não há necessidade do utilizador ter uma password, mas obviamente, caso o utilizador se autentique por *OAuth*, não poderá utilizar o mesmo email para o método `\email` e `\password`.

O utilizador pode possuir 1 único token de sessão, apesar de haver possibilidade de alterar este valor de forma simples. Como é uma relação de composição (losango preenchido), isto significa que o ciclo de vida do Token depende do User, logo se o utilizador for apagado do sistema, os seus tokens de validação perdem sentido e são destruídos.

O utilizador pode ser `\creator` para Trails e `\hiker` para Hikes. Esta relação de composição `\creator` serve para sabermos quem foi o criador do percurso e termos uma ligação para que seja possível apagar trilhos. A relação de composição `\hiker` serve para compreendermos quando um utilizador está ativamente a percorrer um trilho. Tal como o token, caso o utilizador seja apagado, os Hikes e Trails associados ao mesmo também serão.

O utilizador possui um `UserRank` que o categoriza (`\NEW` ou `\VERIFIED`). Esta categorização serve para habilitar o utilizador a poder criar trilhos e a evolução acontece quando o utilizador percorrer 10 caminhos ou 50 quilómetros, tendo como objetivo filtrar e reduzir a possibilidade de utilizadores com más intenções, já que com os recursos disponíveis, não existe possibilidade de fazer validação de percursos.

4.3.1.2 Trail

A classe Trail é a entidade central do domínio geográfico. Representa um percurso predefinido, identificado por um `\id`, um `\name` e constituído por um caminho (`\path` da tipologia `List<Point>`).

O trilho está classificado pela sua dificuldade (`TrailDifficulty`: `\Beginner`, `\Intermediate`, `\Advanced`), categorização alcançada a partir do tamanho do trilho.

Um trilho pode ser `\parentTrail`, que indica a existência de uma ligação entre trilhos, pois caso o utilizador não queira percorrer o trilho completo, pode criar um sub-trilho para percorrer. Caso um trilho tenha um `\parentTrail` que seja apagado, ele vira um trilho simples.

Um trilho tem 2 pontos principais, de classe `Point`, `\start` e `\end` que indicam os extremos do percurso, tendo também uma lista de `Point` que indica todos os pontos necessário o utilizador percorrer para determinar se completou ou não o percurso.

Um trilho "contém" várias caminhadas (* - zero ou mais). Se um trilho deixar de existir, os registos de atividades (Hike) realizadas exclusivamente nesse trilho perdem a sua referência principal, mas continuam a existir no sistema.

4.3.1.3 Hike

O Hike é o que podemos chamar de uma entidade de evento, que representa a ação temporal de um utilizador a percorrer um trilho. Tem um `\id` e a marcação temporal (`\time` do tipo `Instant`).

Uma caminhada, tal como o trilho, também recebe 2 pontos, `\entry` e `\exit`, já que o objetivo é um utilizador poder começar em qualquer extremo do percurso, sem ser preciso haver 2 trilhos, para o mesmo percurso, em direções diferentes.

4.3.2 Modelo ER

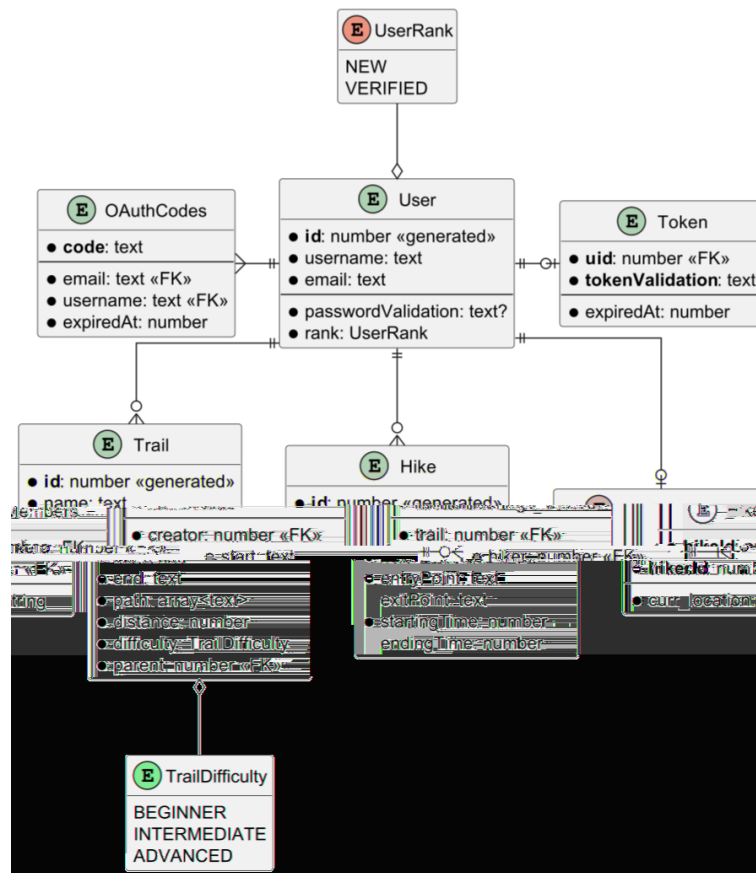


Figura 4.10: UML do Modelo ER desenvolvido

O modelo ER apresentado, figura 4.10 ilustra a adaptação física do modelo de domínio para a arquitetura de base de dados relacional. A essência das três entidades centrais (User, Trail, Hike) mantém-se, mas a estrutura foca-se agora em chaves primárias, determinadas pelos parâmetros a negrito, chaves estrangeiras («FK») e na simplificação de dados complexos.

4.3.2.1 User

A lógica de negócio detalhada na secção 4.3.1.1 (autenticação mista com "passwordValidation" opcional) é visível no esquema físico com o sufixo "?". A restrição de um único token de sessão por utilizador é garantida pela cardinalidade (1 para 0..1) entre a tabela User e Token, onde o Token utiliza o "id" do utilizador (uid) como chave estrangeira e em conjunto com o valor guardado na base de dados, fazem a sua chave primária.

4.3.2.2 Trail e Hike

Para otimizar o desempenho da base de dados, os dados geográficos e temporais foram simplificados para formatos nativos: Em Trail, os pontos `\start` e `\end` são guardados como texto (coordenadas serializadas), e o percurso completo (`\path`) é armazenado num array de texto. A ligação obrigatória ao criador do trilho é assegurada pela chave estrangeira `\creator`, `\id` do utilizador. Em Hike, a mesma lógica aplica-se aos pontos de `\entryPoint` e `\exitPoint`. O atributo de tempo foi desdobrado em `startingTime` e `endingTime` (formato numérico, como timestamps Unix) para facilitar o cálculo da duração da atividade.

4.3.2.3 HikeMembers

A evolução mais significativa neste modelo ER face ao domínio inicial é a adição da tabela associativa `HikeMembers`. Enquanto a tabela `Hike` regista o evento e o seu criador/líder (através da chave estrangeira `\hiker`), a tabela `HikeMembers` resolve a necessidade de termos múltiplos utilizadores na mesma atividade. Ao cruzar o `\id` da caminhada (`\hikeId`) com os `\id`'s dos utilizadores envolvidos (`\hikerId`), o sistema passa a suportar caminhadas em grupo. Adicionalmente, esta tabela inclui o atributo `curr_location`, permitindo monitorizar e persistir a localização individualizada de cada membro do grupo ao longo do percurso.

4.3.2.4 OAuthCodes

A tabela `OAuthCodes` foi introduzida no modelo ER para evitar a dependência de uma solução externa de armazenamento temporário, como o *Redis*, para a validação dos códigos gerados durante o fluxo de *OAuth*.

Os campos `\email` e `\username` são definidos como chaves estrangeiras da tabela `User`. Esta definição não foi imediata: o *Exposed*, ao gerar estas colunas em `User` através de *uniqueIndex*, cria apenas um índice único, e não uma *unique constraint*, reconhecida pelo *PostgreSQL* como válida para servir de alvo de uma chave estrangeira.

Para ser possível relacionar estes campos à tabela `OAuthCodes`, é necessário criar, na inicialização do esquema, as *constraints* `UNIQUE` dos campos da tabela `User`, antes da criação da tabela `OAuthCodes`.

Ao contrário do que sucede em `User`, os campos `\email` e `\username` em `OAuthCodes` não têm uma restrição de unicidade: um mesmo utilizador pode iniciar múltiplos fluxos de *OAuth* em simultâneo (por exemplo, a partir de dispositivos diferentes), pelo que podem legitimamente coexistir vários códigos válidos associados ao mesmo email/username. O campo `\code`, gerado aleatoriamente através de *SecureRandom*, constitui a chave primária da tabela, garantindo a sua unicidade. O campo `\expiredAt` regista o instante de expiração do código, permitindo determinar a sua validade no momento da verificação.

4.3.3 Rotas

A API criada para a finalidade deste projeto disponibiliza rotas destinadas à gestão de utilizadores, trilhos e caminhadas. Também é destinada a rota `\docs`", que apresenta uma página web que documenta as rotas descritas neste documento de forma interativa.

Salvo indicado em contrário, todas as rotas são acessíveis através do método HTTP GET, e formatam as suas respostas de acordo com a especificação JSON. Os tipos de retorno de cada rota apresentada estão associados ao seu sucesso; em caso de erro, qualquer rota responderá com uma mensagem de erro na sua resposta.

É relevante salientar que, nestas rotas, a notação `\{x\}`" indica uma variável de caminho (neste exemplo, denominada `\x`"), necessária para o correto funcionamento da rota.

4.3.3.1 Utilizadores

As rotas apresentadas nesta secção possibilitam a gestão de utilizadores, permitindo ações como criação, autenticação ou eliminação.

- **Rotas sem autenticação**

- `"/users/create"`
 - * Possibilita a criação de um utilizador
 - * **Corpo:** nome, email e palavra-passe; formato JSON
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON
- `"/users/login"`
 - * Possibilita a autenticação de um utilizador
 - * **Corpo:** email e palavra-passe; formato JSON
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON
- `"/users/oauth/code"`
 - * Verifica o código fornecido ao utilizador pelo *deep link* da rota callback do *OAuth*
 - * **Corpo:** email, nome e código; formato JSON
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON

- Rotas com autenticação por OAuth

- “/users/oauth/google”
 - * Redireciona o cliente para a página oficial de *OAuth* da Google
 - * **Retorno:** nenhum porque redireciona
- “/users/oauth/callback”
 - * Recebe o *callback* do *OAuth* e redireciona com um *deep link*
 - * **Path:** \trekio://oauth/callback?code=...&email=...&username=...&new=..."
ou \trekio://oauth/callback?error=..."

- Rotas com autenticação por JWT

- “/users/self”
 - * Fornece os detalhes do próprio utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nome de utilizador e \rank"; formato JSON
- “/users/identifier”
 - * Possibilita a busca dos detalhes de um utilizador pelo seu nome ou pelo id.
 - * **Corpo:** nenhum
 - * **Retorno:** id, nome de utilizador e \rank"; formato JSON
- “/users/{uid}/trails”
 - * Apresenta os trilhos criados pelo utilizador especificado
 - * **Corpo:** nenhum
 - * **Retorno:** lista de trilhos; formato JSON
- “/users/{uid}/stats”
 - * Apresenta as estatísticas pessoais do utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** identificador do utilizador, número de trilhos percorridos, total de quilómetros percorridos, e total de tempo gasto em caminhadas; formato JSON

- Rotas com autenticação por *Bearer token*

- “/users/refresh”

- * Atualiza o token de acesso do utilizador, utilizado em rotas protegidas por JWT
 - * **Corpo:** nenhum
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON

- “/users/update”

- * Pode atualizar tanto o nome como a password da sua conta.
 - * **Corpo:** nenhum
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON

- “/users/logout”

- * Invalida os tokens gerados para o utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nenhum

- “/users/delete”

- * Elimina todos os dados pessoais do utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nenhum

4.3.3.2 Trilhos

As rotas aqui apresentadas possibilitam a consulta e gestão de trilhos, permitindo ações como criação, importação ou eliminação. Para estas rotas, foi aplicada a autenticação por JWT.

- “/trails/create”
 - Possibilita a criação de um trilho.
 - **Corpo:** nome, pontos inicial e final, lista de pontos intermédios, dificuldade e identificador do trilho parente; formato JSON
 - **Retorno:** identificador do trilho; formato JSON
- “/trails/import”
 - Possibilita a importação de um trilho em formato KML
 - **Corpo:** ficheiro KML contendo pontos inicial e final, e lista de pontos intermédios; formato KML/XML, suportado por *MIME types* "application/xml", "text/xml" ou "application/vnd.google-earth.kml+xml"
 - **Retorno:** identificador do trilho; formato JSON
- “/trails/available”
 - Apresenta uma lista paginada de trilhos disponíveis, sendo possível navegar entre páginas através dos parâmetros numéricos de pesquisa "skip" e "limit"
 - **Corpo:** nenhum
 - **Retorno:** lista paginada de trilhos; formato JSON
- “/trails/{tid}” (método GET)
 - Apresenta os detalhes de um trilho pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nome, pontos inicial e final, lista de pontos intermédios, dificuldade e identificador do trilho parente; formato JSON
- “/trails/{tid}” (método PUT)
 - Permite a atualização dos detalhes de um trilho pelo seu identificador
 - **Corpo:** nome novo do trilho
 - **Retorno:** nenhum
- “/trails/{tid}” (método DELETE)
 - Elimina um trilho pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nenhum

- “/trails/{tid}/start” (protocolo *WebSockets*)
 - Sinaliza o início de uma caminhada num trilho pelo seu identificador; como utiliza o protocolo *WebSockets*, reutiliza o mesmo canal de comunicação para realizar o acompanhamento da caminhada em tempo real pelo cliente e pelo servidor
 - **Corpo:** localização atual do cliente, enviada periodicamente em tempo real
 - **Retorno:** localização atual de todos os caminhantes a frequentar o trilho, incluindo o utilizador atual; formato JSON

4.3.3.3 Caminhadas

As rotas aqui apresentadas possibilitam a consulta, finalização e cancelamento de caminhadas. Para estas rotas, foi aplicada a autenticação por JWT.

- “/hikes/{hid}” (método GET)
 - Apresenta os detalhes de uma caminhada pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** identificadores de caminhada, caminhante e trilho, pontos de entrada e saída, e instantes de início e fim da caminhada; formato JSON

4.3.4 *Load Balancing* Dinâmico e *Deployment*

O *deployment* do servidor é bastante importante num projeto com alvo Android, pois este não tem capacidade de executar o servidor na mesma máquina (*localhost*). Foi necessário fazer uma escolha entre 2 tecnologias já utilizados no curso: *ngrok* ou *Render*.

O *ngrok* força o *deployment* manual, mas tem uma vantagem, consegue coexistir com o *nginx* que possibilita *load balancing* dinâmico. Por outro lado, o *Render* fornece, no plano gratuito, apenas *load balancing* estático, mas tem *deployment* automático, tendo suporte também para a base de dados e *Redis*.

Mas, tendo em conta que um dos objetivos primários deste projeto foi garantir *load balancing* dinâmico, algo não alcançado pelo grupo num projeto anterior, seguiu-se com o conjunto *nginx* e *ngrok*.

4.3.5 Docker e Automação do Ambiente

Todo o ecossistema do *Backend* foi containerizado através do Docker Compose. A configuração base define quatro serviços fundamentais: a base de dados PostgreSQL, o Redis (como intermediário Pub/Sub e *caching*), a aplicação Ktor (JVM) e o proxy *nginx*. A infraestrutura tira partido de *healthchecks*, garantindo que a aplicação apenas inicia a sua execução quando a base de dados se encontrar totalmente pronta a receber ligações. Como a arquitetura foi desenhada nativamente para suportar *load balancing*, o ambiente Docker foi configurado para escalabilidade horizontal. Através do argumento de *scale* do Docker Compose, é possível levantar dinamicamente múltiplas instâncias da JVM, as quais são automaticamente reconhecidas e geridas pelo *nginx*.

Para garantir uma elevada produtividade durante a fase de desenvolvimento, a orquestração destes contentores foi integrada diretamente no Gradle (*build system* do projeto). Foram desenvolvidas tarefas personalizadas, como `\dockerUp`", `\dockerDown`" e `\waitForDatabase`", que abstraem a complexidade do ambiente local. Na prática, estas tarefas encarregam-se de compilar o código, carregar variáveis de ambiente (através de um ficheiro `\.env`) e iniciar os serviços necessários com um único comando. Atualmente, a execução de testes unitários e a execução normal da aplicação partilham o mesmo contentor de base de dados, pelo que não é possível correr ambos em simultâneo. Esta é uma limitação identificada para trabalho futuro, nomeadamente através do isolamento da base de dados de testes num contentor dedicado.

Capítulo 5

Conclusões

Tendo em conta os diversos problemas e desafios encontrados ao longo do desenvolvimento desta aplicação, a versão final apresenta um número de funcionalidades inferior ao inicialmente previsto. Em particular, não foi possível implementar a integração de uma LLM para apoio à navegação do utilizador dentro da aplicação, nomeadamente na execução de ações como a criação ou a procura de trilhos através de linguagem natural. Adicionalmente, não foram implementadas as funcionalidades classificadas como opcionais desde o início do projeto, entre as quais a avaliação de condições meteorológicas, um sistema de amigos, o cálculo e apresentação da velocidade média ao longo de uma caminhada, a sugestão de trilhos com base no histórico de caminhadas do utilizador e o suporte para verificação do email usado na criação de uma conta por email/nome/palavra-chave. Estas limitações resultaram sobretudo de restrições de tempo face à complexidade das funcionalidades essenciais, assim como da dificuldade na resolução dos diversos problemas na implementação do alvo Web.

A nível tecnológico, a escolha do KMP revelou-se simultaneamente uma vantagem e um desafio. Por um lado, permitiu a partilha efetiva de código e lógica de negócio entre *frontend* e *backend*, tal como descrito na secção 3.1.1. Por outro lado, a reduzida maturidade do ecossistema, especialmente para a Web, cuja dependência de bibliotecas de terceiros (como o *MapBox GL JS*, por falta de opção KMP), impossibilitou a conclusão deste alvo dentro do prazo do projeto, e por consequência, reduzindo o tempo disponível para trabalhar sobre o alvo Android. A ausência de equipamento adequado impediu igualmente o desenvolvimento do alvo iOS, apesar de a arquitetura ter sido pensada desde o início para o suportar no futuro.

Do ponto de vista da equipa, o desenvolvimento deste projeto permitiu consolidar competências técnicas em diversas áreas:

- A nível de *backend* destaca-se:
 - A aprendizagem do Ktor e do desenvolvimento sobre um servidor não bloqueante, alternativa competitiva face a soluções mais estabelecidas como o Spring Boot (secção 3.2);
 - O Exposed, que se revelou, na perspetiva do programador, superior ao JDBC e ao JDBI, evidenciando também as dificuldades inerentes ao acesso a dados não bloqueante;
 - O funcionamento conjunto de WebSockets e Redis na garantia do objetivo de escalabilidade horizontal; (secção 3.3.3);
- A nível de *frontend*:
 - O aprofundamento do funcionamento do Android, nomeadamente na gestão de permissões de localização, alteração de idioma e receção de *deep links*;
 - A implementação de autenticação via OAuth; (secção 3.3.1.1);
- Em torno de todo o ecossistema KMP, a perceção do quão vantajosa esta tecnologia é para o desenvolvimento *mobile*, ainda que exija especial cautela quando aplicada à Web, dada a volatilidade das bibliotecas disponíveis para este alvo. (secção 3.1.1);

Para além das competências técnicas específicas, o projeto contribuiu também para uma maturidade acrescida a nível de processo de desenvolvimento. O curso, ao longo do seu percurso, não impõe a necessidade de estruturação de projetos de raiz, focando-se sobretudo na realização de projetos já delineados. Assim, este trabalho evidenciou o tempo e esforço reais que a estruturação e planeamento de um projeto de maior escala exigem e o quanto impactam o resultado final do projeto.

Esta consciencialização estendeu-se a boas práticas de desenvolvimento:

- A importância de nomes descritivos para funções, variáveis e parâmetros;
- A extrema importância de desenvolver funções mais curtas e objetivas, facilitando a sua compreensão e do restante código;
- A importância de evitar a duplicação de código, através da sua fatorização em funções reutilizáveis, pois caso não seja feito, aumenta o risco de levar a *bugs* com mudanças no código;
- A importância da existência de registo de logs ao longo de toda a aplicação, tanto em *backend* como *frontend*;
- A necessidade de testes unitários, principalmente em projetos de maior densidade e complexidade;
- A importância da documentação de uma biblioteca pública, pois dificulta muito o seu uso quando esta está em falta. (Infelizmente a biblioteca realizada pelo grupo não tem a melhor documentação devido à falta de tempo e priorização do restante projeto)

Adicionalmente, o projeto permitiu à equipa familiarizar-se com ferramentas de automação e distribuição até então desconhecidas:

- *GitHub Workflows* e *GitHub Secrets*;
- *Badges* para um README mais informativo e dinâmico;
- O processo de publicação no Maven Central;
- A utilização de ficheiros `\".env\"`, incluindo a sua adaptação dinâmica através de *workflows* do GitHub, abordagem que, por falta de tempo, não chegou a ser replicada na adaptação automática do próprio README do projeto.

Como trabalho futuro, identificam-se como prioritários a conclusão do alvo Web, a exploração do alvo iOS, e a implementação da LLM e das funcionalidades opcionais anteriormente referidas.

Referências

- [1] JetBrains. Kotlin multiplatform. <https://www.jetbrains.com/kotlin-multiplatform/>, 2026. Acedido: Jun. 1, 2026.
- [2] JetBrains. Ktor. <https://ktor.io>, 2026. Acedido: Jun. 1, 2026.
- [3] JetBrains. Exposed. <https://www.jetbrains.com/exposed/>. Acedido: Jun. 1, 2026.
- [4] r2dbc. r2dbc. <https://github.com/r2dbc/r2dbc-pool>. Acedido: Jul. 22, 2026.
- [5] The PostgreSQL Global Development Group. Postgresql. <https://www.postgresql.org>. Acedido: Jun. 8, 2026.
- [6] Redis Ltd. Redis. <https://redis.io>. Acedido: Jun. 1, 2026.
- [7] Will Glozer and Mark Paluch. Lettuce: Advanced java redis client. <https://github.com/redis/lettuce>. Acedido: Jun. 8, 2026.
- [8] Mapbox. Mapbox android. <https://docs.mapbox.com/android/maps/guides/>. Acedido: Jun. 1, 2026.
- [9] Mapbox. Mapbox gl js. <https://docs.mapbox.com/mapbox-gl-js/guides/>. Acedido: Jun. 1, 2026.
- [10] JetBrains. Compose multiplatform. <https://www.jetbrains.com/compose-multiplatform/>. Acedido: Jun. 1, 2026.
- [11] Touchlab. Kermit: Kotlin multiplatform logging. <https://kermit.touchlab.co>. Acedido: Jun. 8, 2026.
- [12] Yasuhiro SHIMIZU. Buildkonfg: Kotlin multiplatform for injecting properties in build-config. <https://github.com/yshrmz/BuildKonfig>. Acedido: Jun. 8, 2026.
- [13] Jonathan Leitschuh. Ktlint: Wrapper plugin around ktlint. <https://github.com/JLLeitschuh/ktlint-gradle>. Acedido: Jun. 8, 2026.
- [14] Niklas Baudy. Gradle maven publish plugin. <https://github.com/vanniktech/gradle-maven-publish-plugin>. Acedido: Jun. 8, 2026.

- [15] Sonatype. Maven central. <https://central.sonatype.com>. Acedido: Jun. 8, 2026.
- [16] SmartBear Software. Swagger. <https://swagger.io>. Acedido: Jun. 8, 2026.
- [17] F5 Inc. Nginx. <https://nginx.org>. Acedido: Jun. 1, 2026.
- [18] Docker Inc. Docker. <https://www.docker.com>, 2026. Acedido: Jun. 1, 2026.
- [19] Gradle Inc. Gradle. <https://gradle.org>. Acedido: Jun. 1, 2026.
- [20] ngrok, Inc. ngrok. <https://ngrok.com>. Acedido: Jul. 22, 2026.
- [21] OAuth Community. Oauth 2.0. <https://oauth.net/2/>. Acedido: Jun. 1, 2026.
- [22] OpenID Foundation. Openid connect. <https://openid.net/connect/>. Acedido: Jun. 1, 2026.
- [23] Mozilla Developer Network. Websockets api. https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API. Acedido: Jun. 1, 2026.
- [24] Google. Android studio. developer.android.com/studio. Acedido: Jun. 8, 2026.
- [25] React Foundation. React. <https://react.dev>. Acedido: Jun. 8, 2026.
- [26] OpenStreetMap Foundation. Openstreetmap. <https://www.openstreetmap.org>. Acedido: Jun. 1, 2026.
- [27] Google. Google maps platform. <https://developers.google.com/maps>. Acedido: Jun. 1, 2026.
- [28] MapLibre. Maplibre. <https://maplibre.org>. Acedido: Jun. 1, 2026.
- [29] VMware. Spring boot. <https://spring.io/projects/spring-boot>. Acedido: Jun. 1, 2026.
- [30] VMware. Spring framework. <https://spring.io/projects/spring-framework>. Acedido: Jun. 1, 2026.
- [31] Oracle. Java Platform. Java database connectivity (jdbc). <https://docs.oracle.com/en/database/oracle/oracle-database/26/jjdbc/toc.htm>. Acedido: Jun. 8, 2026.
- [32] Jdbi Community. Java database interface (jdbi). <https://jdbi.org>. Acedido: Jun. 8, 2026.